Árvore

Árvore se trata de um modelo abstrato de uma estrutura de dados hierárquica. Essa estrutura é composta por relacionamentos entre nós de pai e filho. Possui um nó raiz, que possui nós filhos, mas não possui pai.

Algumas aplicações de árvores são:

- Sistemas de arquivos
- Representação de documentos (título, capítulo...)
- Akinator
- Expressões aritméticas

Terminologia

- Raiz (root) - nó que não possui pai
- Nó interno - nó com, pelo menos, um filho
- Nó externo - nó sem filhos
- Ancestral de um nó - pai, avô, bisavô
- Profundidade de um nó - número de ancestrais
- Altura de uma árvore - profundidade máxima
- Descendente de um nó - filho, neto, bisneto

TAD árvore

Métodos genéricos

- integer size( ) - retorna o número de nós da árvore
- integer height( ) - retorna a altura da árvore
- boolean isEmpty( ) - indica se a árvore não possui nós
- Iterator elements( ) - percorrer os elementos da árvore

- Iterator nos( ) - percorrer os nós da árvore

Métodos de acesso

- No root( ) - retorna o nó raiz
- No parente(No) - retorna o nó pai do parâmetro
- Iterator children(No) - percorre os filhos do nó parâmetro

Métodos de consulta

- boolean isInternal(No) - indica se o nó é interno
- boolean isExternal(No) - indica se o nó é externo
- boolean isRoot(No) - indica se o nó é raiz
- integer depth(No) - retorna a profundidade de um nó

Profundidade e altura

A profundidade de um nó pode ser definida recursivamente. Se v for raiz, então a profundidade é 0. Senão, a profundidade é 1 mais a profundidade do pai de V.

```
Algoritmo depth(v)
    se (isRoot(v))
      retorne 0
    senão
      retorne 1+depth(parent(v))
```

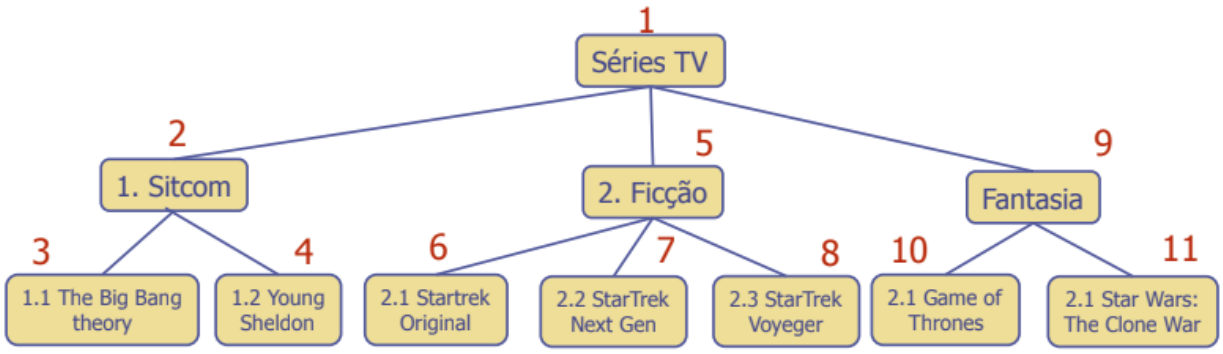
A altura de um nó também pode ser definida recursivamente. Se v for externo, então a altura é 0. Senão, a altura é 1 mais a maior altura de um filho de v.

```
Algoritmo altura(v) – O(n)
    se (isExternal(v))
      retorne 0
    senão
      h=0
      para cada w em children(v)
          h=max(h,altura(w))
      retorne 1+h
```

Travessia pré ordem

Uma travessia visita os nós de uma árvore de uma forma sistemática. Em uma travessia pré ordem, são visitados os pais e, posteriormente, os seus filhos são visitados.

```
Algoritmo preOrder(v)
    visite(v)
    para cada filho w de v
        preorder (w)
```



Nesse caso, começamos pelo root e depois seus filhos. O seu primeiro filho visitado é o **sitcom.** Ao visitá-lo, vamos visitar seus dois filhos também, então a

ordem ficará **Séries TV → Sitcom → The Big Bang Theory → Young Sheldon**. Terminamos de visitar os filhos do primeiro filho do root, então vamos visitar o segundo filho de root, **ficção**. Da mesma forma, vamos visitar os filhos de ficção, e ao terminar, vamo visitar **fantasia** e seus filhos.

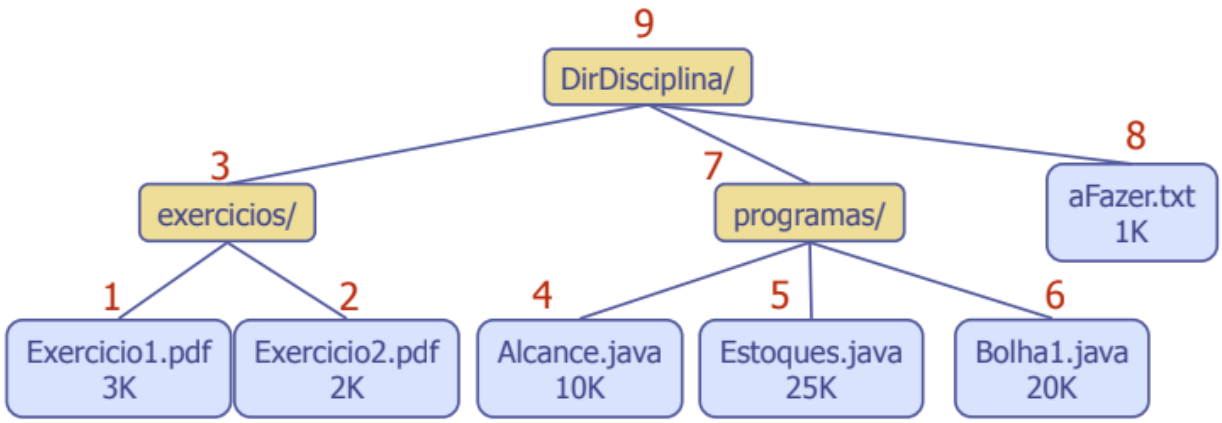
A ordem fica:

Séries TV → Sitcom → The Big Bang Theory → Young Sheldon → Ficção → Startrek original → Startrek Next Gen → Startrek Voyeger → Fantasia → Game of Thrones → Star Wars

Travessia pós ordem

Em uma travessia pós ordem, um nó é visitado depois dos seus filhos terem sido visitados.

Algoritmo *postOrder(v)*
  para cada filho *w* of *v*
    postOrder (w)
  visite(v)



Nesse caso, o root vai ser o último visitado, porque ele só pode ser visitado depois que todos os seus descendentes forem visitados.

Árvore binária

Uma árvore binária é uma árvore que têm, no máximo, dois filhos por nó. Pode ser aplicada em expressões aritméticas e em sistemas de decisão. Os filhos de um nó são um par.

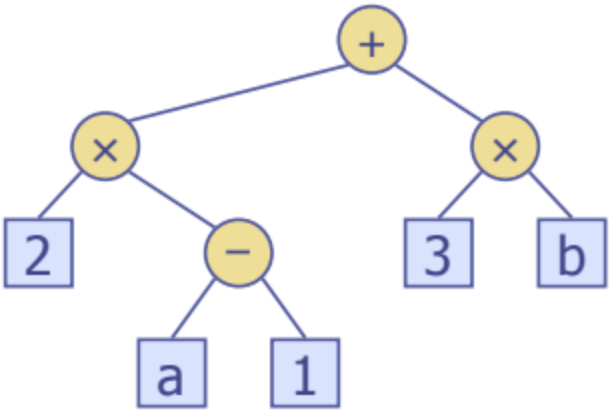
Uma árvore binária **própria** é aquela em que cada nó possui zero ou dois filhos.

Chamamos os filhos de um nó de filho da **esquerda** e filho da **direita.**

Árvore de expressões aritméticas

Árvore binária associada com uma expressão aritmética.

- Nós internos: operadores
- Nós externos: operandos



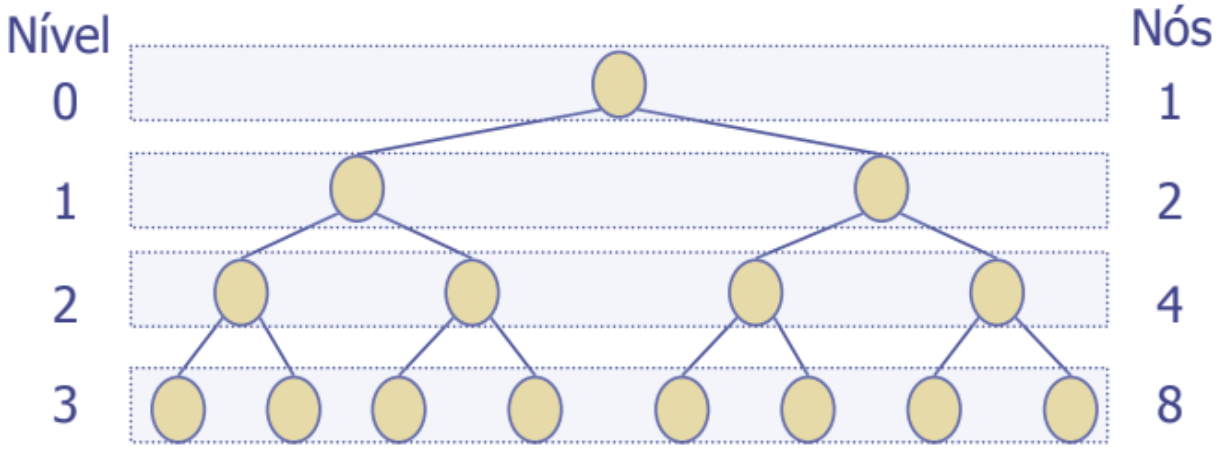
Propriedades

Notação:

- n - número de nós
- e - número de nós externos
- i - número de nós internos
- h - altura

Propriedades:

- e = i + 1
- n = 2e - 1
- h ≤ i
- h ≤ (n - 1)/2
- e ≤ 2^h



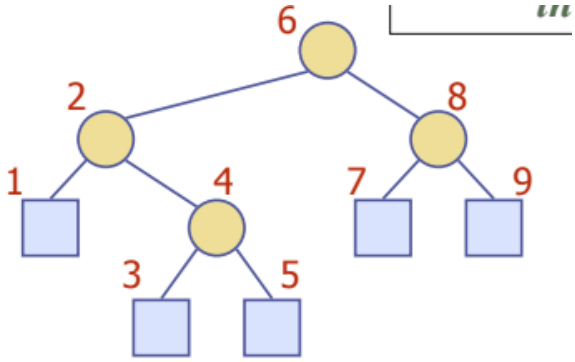
Métodos

- No leftChild(v) - retorna o filho esquerdo de v
- No rightChild(v) - retorna o filho direito de v
- boolean hasLeft(v) - retorna se v tem filho esquerdo
- boolean hasRight(v) - retorna se v tem filho direito

Travessia em ordem

Na travessia em ordem, um nó é visitado depois do filho da esquerda e antes do filho da direita.

```
Algoritmo inOrder(v)
    se (isInternal (v))
        inOrder (leftChild (v))
    visite(v)
    se ( isInternal (v))
        inOrder (rightChild (v))
```



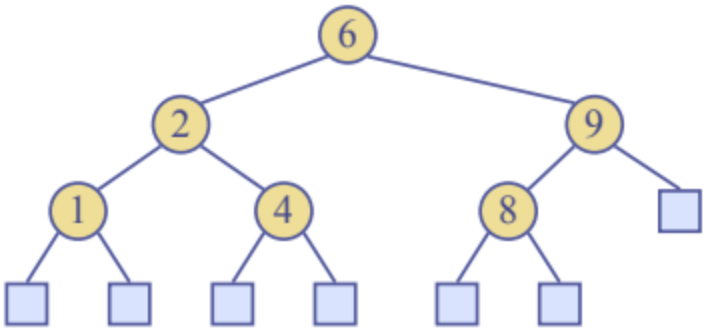
Pode ser aplicado para desenhar uma árvore binária em terminal.

Árvore de pesquisa binária

Uma árvore de pesquisa binária é uma árvore binária que obedece as seguintes propriedades:

- O nó filho da esquerda deve ser menor que o pai
- O nó filho da direita deve ser maior que o pai

Dessa forma, uma travessia em ordem visita as chaves em ordem crescente.

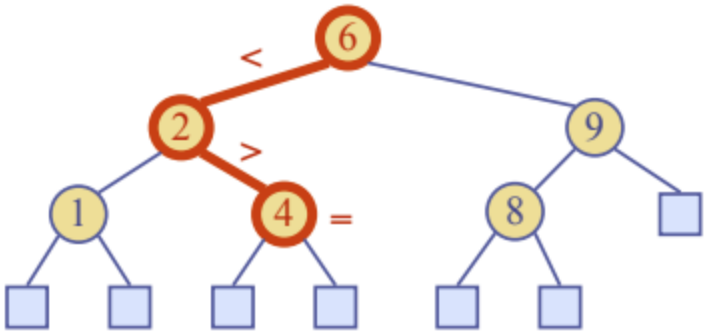


Busca

Para procurar um nó, procuramos a partir da raiz. O próximo nó depende da comparação da nó pesquisado com o nó atual.

Exemplo: `find(4)`

```
Algoritmo TreeSearch(k, v)
    se T.isExternal (v)
        retorne v
    se k < key(v)
        retorne TreeSearch(k, T.left(v))
    senão se k = key(v)
        retorne v
    senão { k > key(v) }
        retorne TreeSearch(k, T.right(v))
```

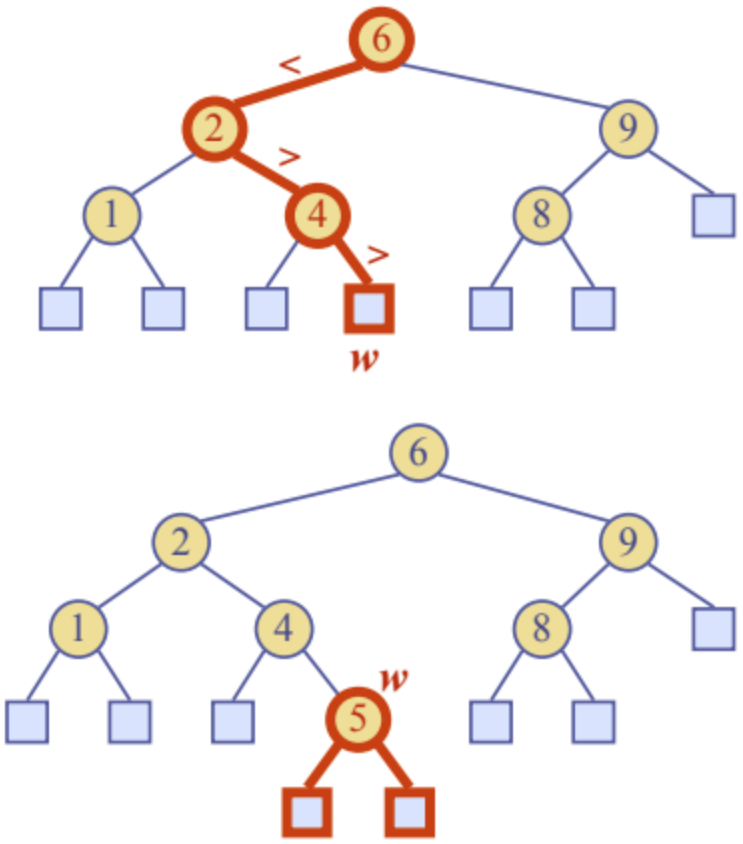


4 é menor do que 6, então vamos buscar a partir do nó esquerdo (menor) de 6. 4 é maior do que 2, então vamos procurar a partir do filho direito (maior) de 2.

Inserção

Assumindo que o número que será inserido ainda não está na árvore, buscamos por uma folha, ou seja, um lugar em que o nó possa ser inserido.

Exemplo: `insert(5)`



5 é menor do que 6 (raiz), então buscamos a partir do menor filho de 6 (esquerdo). 5 é maior do que 2, então buscamos a partir do maior filho de 2 (direito). 5 é maior do que 4, mas 4 não possui filhos, então adicionamos o 5 como filho direito de 4, já que ele é maior.

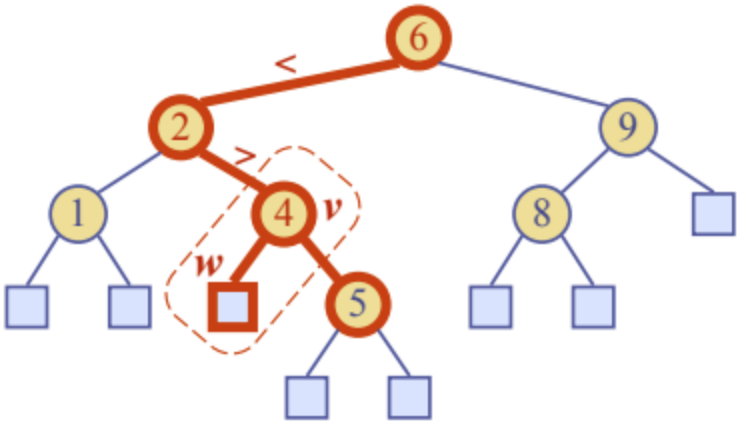
Remoção

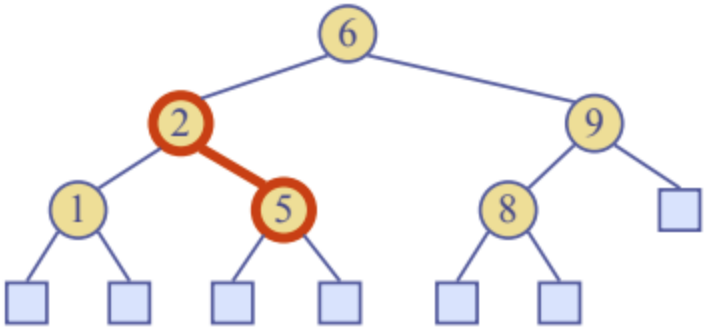
Há 3 casos quando ocorre a remoção:

- O nó não possui filhos
- O nó possui um filho
- O nó possui dois filhos

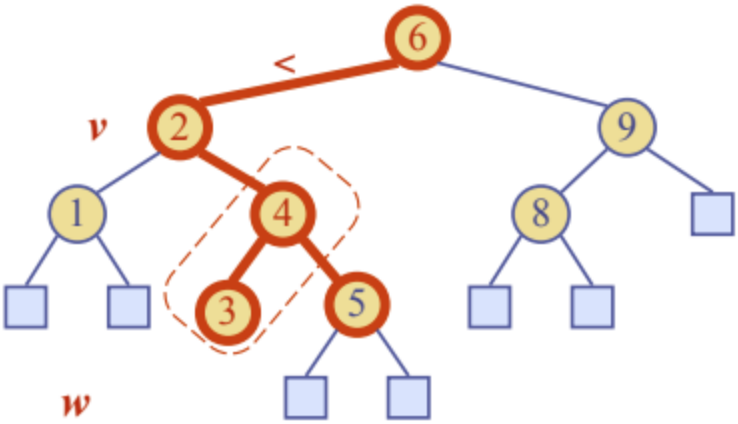
Para o primeiro caso, basta apenar apagar o nó, dessa forma, o seu nó pai terá mais uma folha livre.

No segundo caso, o nó pai do nó que vai ser removido, se tornará pai do filho do nó removido.



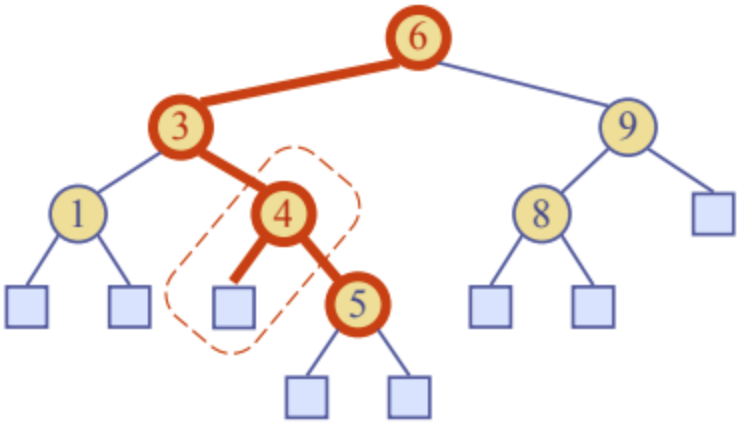


Já no terceiro caso, precisamos fazer uma troca de valores.



Se queremos remover o nó 2, temos que buscar por um nó maior do que o filho esquerdo de 2 e menor do que o filho direito de 2, nesse caso, seria o 3.

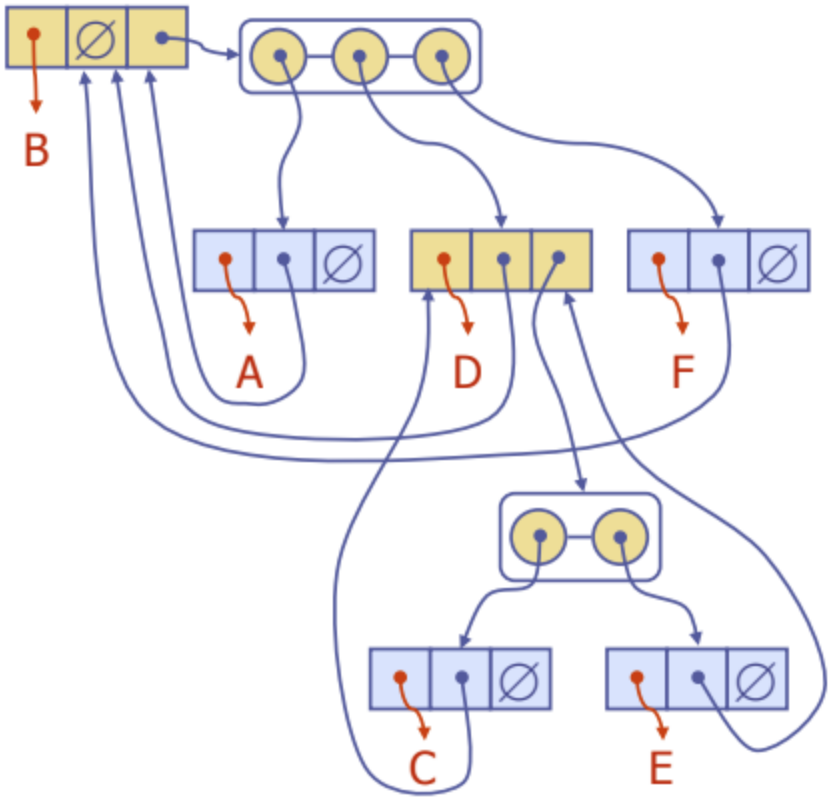
Então, iremos fazer com que o valor do nó 2 se torne 3, e o filho esquerdo de 4 ficará livre.



Estrutura de dados para árvores

Um nó será uma classe que armazena:

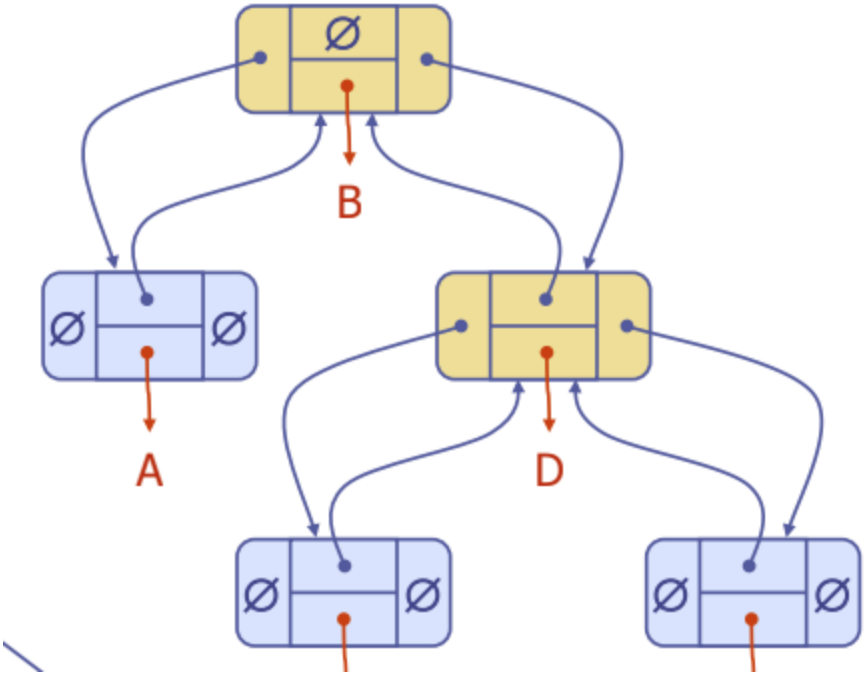
- Elemento
- Nó pai
- Nós filhos (sequência, vector, array, pair, etc)



Estrutura de dados para árvore binária

Um nó irá armazenar:

- Elemento
- Nó pai
- Nó filho da esquerda
- Nó filho da direita



Código - Árvore binária de pesquisa

```
import java.util.ArrayList;

class ArvoreBinaria {
    private No raiz;
    private Integer tam;

    private ArrayList<No> visitas;
    private No no_inorder;

    public ArvoreBinaria(No raiz){
        tam = 1;
        this.raiz = raiz;
    }
    public Integer size(){
        return tam;
    }
    public Boolean isEmpty(){
```

```
        if(tam == 0) return true;
        return false;
    }
    public No root(){
        return raiz;
    }
    public No parent(No node){
        return node.getPai();
    }
    public No leftChild(No node){
        return node.getFilho_esquerda();
    }
    public No rightChild(No node){
        return node.getFilho_direita();
    }
    public Boolean isLeftChild(No no){
        if(no.getPai().getFilho_esquerda() == no) return true;
        return false;
    }
    public Boolean isRightChild(No no){
        if(no.getPai().getFilho_direita() == no) return true;
        return false;
    }
    public Boolean isExternal(No node){
        if(node.getFilho_direita() == null && node.getFilho_esqu
        return false;
    }
    public Boolean isInternal(No node){
        if(!isExternal(node)) return true;
        return false;
    }
    public Boolean isRoot(No node){
        if(node == raiz) return true;
        return false;
    }
    public Boolean hasLeftChild(No node){
```

```
        if(node.getFilho_esquerda() != null) return true;
        return false;
    }
    public Boolean hasRightChild(No node){
        if(node.getFilho_direita() != null) return true;
        return false;
    }
    public Integer height(No o){
        if(isExternal(o)) return 0;
        else{
            Integer max = 0;
            Integer h = height(o.getFilho_direita())+1;
            Integer h2 = height(o.getFilho_esquerda())+1;
            if(h > max) max = h;
            if(h2 > max) max = h2;
            return max;
        }
    }
    public Integer depth(No o){
        if(o == raiz) return 0;
        return 1 + depth(o.getPai());
    }
    public void insertNode(No new_node){
        No node = raiz;
        while(new_node.getPai() == null){
            if(new_node.getElemento() > node.getElemento()){
                if(node.getFilho_direita() == null){
                    node.setFilho_direita(new_node);
                    new_node.setPai(node);
                }
                else
                    node = node.getFilho_direita();
            }
            else if(new_node.getElemento() < node.getElemento()
                if(node.getFilho_esquerda() == null){
                    node.setFilho_esquerda(new_node);
```

```
                    new_node.setPai(node);
                }
                else
                    node = node.getFilho_esquerda();
            }
        }
        tam++;
    }
    public Boolean searchNode(No node_search, No node){
        if(isExternal(node_search)) return true;
        while(node != node_search && node != null){
            if(node_search.getElemento() > node.getElemento()){
                node = node.getFilho_direita();
            }
            else if(node_search.getElemento() < node.getElemento
                node = node.getFilho_esquerda();
            }
        }
        if(node == null) return false;
        return true;
    }
    public void removeNode(No node_removed){
        //nó sem filhos
        if(isExternal(node_removed)){
            if(isLeftChild(node_removed)) node_removed.getPai()
            else node_removed.getPai().setFilho_direita(null);
            node_removed = null;
            tam--;
        }
        //nó com um filho
        else if(node_removed.oneChild()){
            No node_pai = node_removed.getPai();
            if(isLeftChild(node_removed)){
                if(node_removed.getFilho_direita() != null) node
                else node_pai.setFilho_esquerda(node_removed.get
            }
```

```
            else{
                if(node_removed.getFilho_direita() != null) node
                else node_pai.setFilho_direita(node_removed.getF
            }
            tam--;
        }
        //nó com dois filhos
        else{
            No node = node_removed.getFilho_direita();
            No node_pai = node_removed.getPai();
            No node_sub = inOrder_return(node);
            node_sub.setFilho_esquerda(node_removed.getFilho_esq
            if(isLeftChild(node_removed)){
                node_pai.setFilho_esquerda(node_sub);
            }
            else{
                node_pai.setFilho_direita(node_sub);
            }
            tam--;
        }
    }

    //visitas
    private void preOrder_func(No o){
        visitas.add(o);
        if(o.getFilho_esquerda() != null) preOrder_func(o.getFil
        if(o.getFilho_direita() != null) preOrder_func(o.getFilh
    }
    public ArrayList<No> preOrder_print(){
        visitas = new ArrayList<No>();
        preOrder_func(raiz);
        return visitas;
    }
    public void posOrder_func(No o){
        if(o.getFilho_esquerda() != null) posOrder_func(o.getFil
        if(o.getFilho_direita() != null) posOrder_func(o.getFilh
```

```
        visitas.add(o);
    }
    public ArrayList<No> posOrder_print(){
        visitas = new ArrayList<No>();
        posOrder_func(raiz);
        return visitas;
    }
    private void inOrder_func(No o){
        if(isInternal(o) && o.getFilho_esquerda() != null)
            inOrder_func(o.getFilho_esquerda());
        visitas.add(o);
        if(isInternal(o) && o.getFilho_direita() != null)
            inOrder_func(o.getFilho_direita());
    }
    public ArrayList<No> inOrder_print(){
        visitas = new ArrayList<No>();
        inOrder_func(raiz);
        return visitas;
    }
    private void inOrder_visite(No o, Boolean break_rec){
        if(isInternal(o) && o.getFilho_esquerda() != null)
            inOrder_func(o.getFilho_esquerda());
        if(break_rec){
            return;
        }
        if(o.oneChild() || isExternal(o)){
            no_inorder = o;
            break_rec = true;
        }
        if(isInternal(o) && o.getFilho_direita() != null)
            inOrder_func(o.getFilho_direita());
    }
    private No inOrder_return(No o){
        no_inorder = new No();
        inOrder_visite(o, false);
        return no_inorder;
```

```
    }
    //DESENHAR ÁRVORE
        //matriz: altura max por tamanho
    public void desenharArvore(){
        Integer altura = height(raiz);
        Integer[][] matriz = new Integer[altura+1][size()];
        ArrayList<No> nodes = inOrder_print();
        Integer k = 0;
        for(No node : nodes){
            matriz[depth(node)][k] = node.getElemento();
            k++;
        }
        for(int i = 0; i <= altura; i++){
            for(int j = 0; j < size(); j++){
                if(matriz[i][j] == null)
                    System.out.print(" ");
                else
                    System.out.print(matriz[i][j]);
                System.out.print(" ");
            }
            System.out.println();
        }
    }
}
```